International University

School of Computer Science and Engineering

Artificial Intelligence

Lab 03: Vacuum Agent

Semester: 1, Year: 2025 - 2026
Lab Instructor: Dr. Ly Tu Nga

Full name: Võ Trí Khôi
Student ID: ITCSIU24045
Group: G01

1. Introduction

This report presents the implementation and evaluation of a vacuum agent that uses Greedy Best-First Search and A* Search algorithms to explore and clean an unknown grid environment. The agent starts from a random position, explores all reachable tiles, sucks up dirt, and returns to its home position at (1, 1).

1.1 Performance Scoring

The agent's performance is measured by a score with the following rules:

- Initial score: −1000
- Sucking dirt: +100 points
- Shutting down at home: +1000 points
- All other actions (forward, turn left, turn right, no-op): −1 point

1.2 GUI Preview

Here is a snapshot of the agent running in the environment, with the visual grid on the left and the action log on the right:

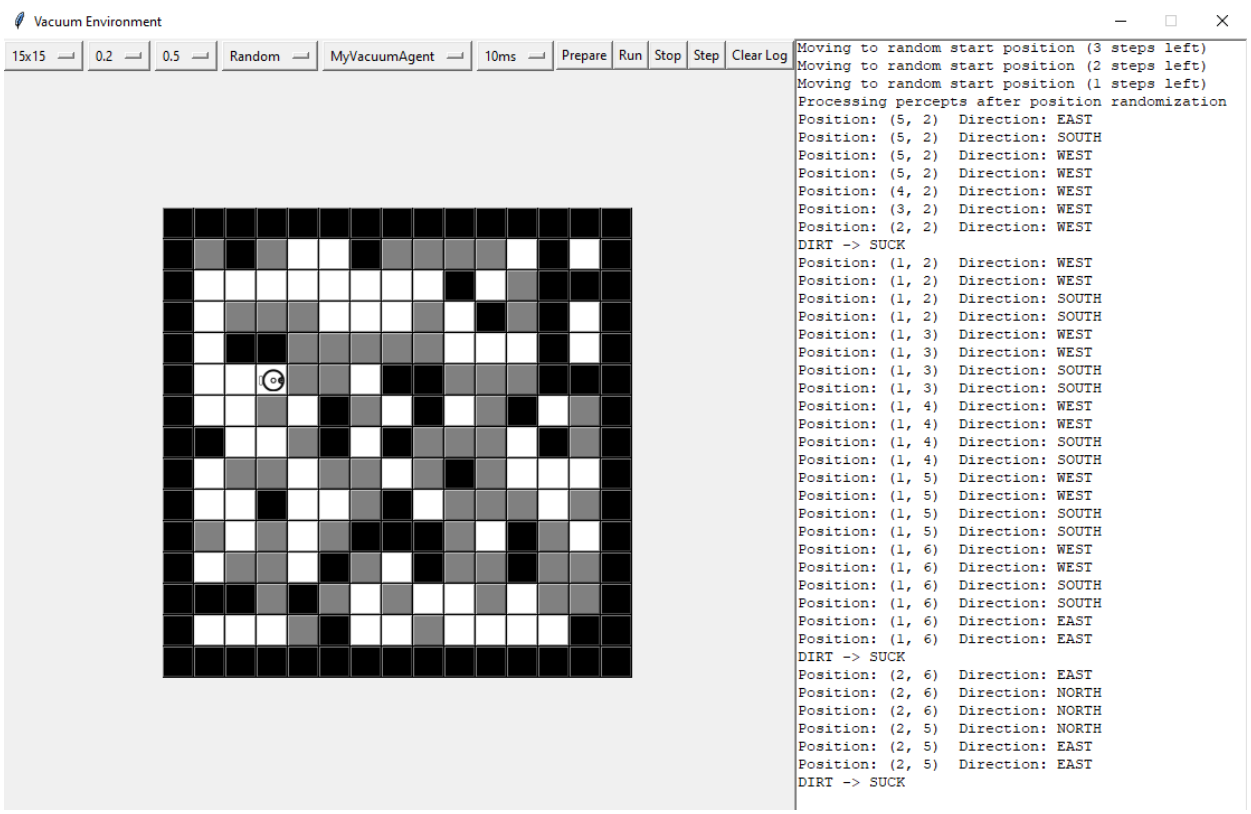

Figure 1.2.1: GUI Preview – Agent running

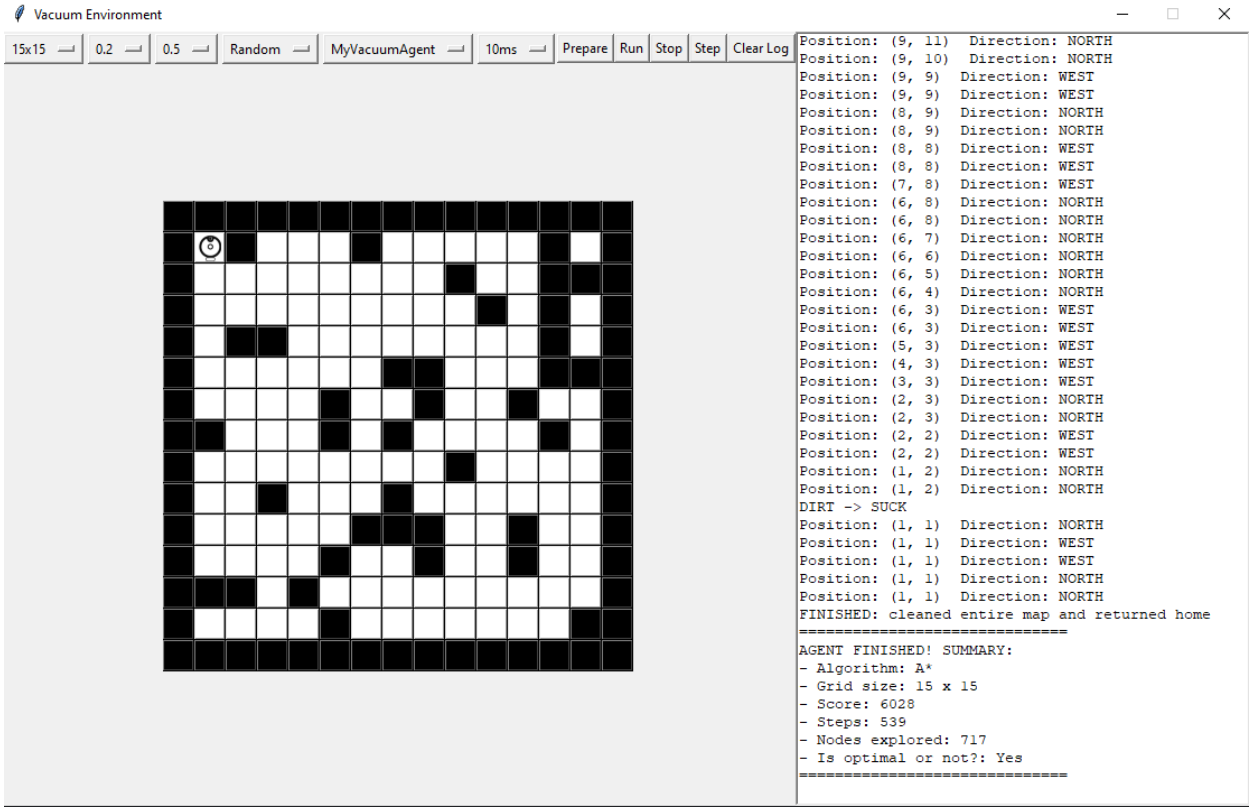

Figure 1.2.2: GUI Preview – Agent finished

2. Implementation

2.1 Agent Architecture

The agent is implemented in myvacuumagent.py and consists of two main classes:

- MyAgentState: Maintains the agent's internal model of the world, including a 2D grid where each cell can be UNKNOWN, WALL, CLEAR, DIRTY, or HOME.
- MyVacuumAgent: The agent itself, which uses search algorithms to decide its next action.

2.2 Agent Decision Flow

Each time the execute() method is called, the agent follows this logic:

1. Initialization Phase (first 11 steps): The agent moves to a random start position using move_to_random_start_position().
2. Perception: The agent reads percepts — bump, dirt, and home — and updates its internal world model.
3. Clean: If the current tile is dirty, the agent sucks the dirt (+100 score).
4. Plan: If no route is planned, the agent runs Greedy Best-First Search or A* Search to find the nearest unknown tile.

5. Return Home: If no unknown tiles remain, the agent uses the same search algorithm to navigate back to the home position (1, 1).
6. Shutdown: Once the agent is at home and all tiles are explored, it shuts down (+1000 score).
7. Execute Plan: If a route exists, the agent executes the next move (turn or forward).

2.3 Heuristic Function

Both algorithms share the same heuristic function h(n), which uses the Manhattan distance:

```
def h(cx, cy):
    if return_home:
        hx, hy = self.home_pos
        return abs(cx - hx) + abs(cy - hy)
    else:
        if not unknowns: return 0
        return min(abs(cx - ux) + abs(cy - uy) for ux, uy in unknowns)
```

- Exploration mode (return_home=False): h(n) = minimum Manhattan distance from node n to any unknown tile. This guides the agent toward the closest unexplored area.
- Return home mode (return_home=True): h(n) = Manhattan distance from node n to the home position (1, 1).

The Manhattan distance is admissible (never overestimates the true cost) and consistent (satisfies the triangle inequality), since the agent can only move in 4 cardinal directions and each move costs 1.

2.4 Greedy Best-First Search Implementation

```
def best_first_search(self, return_home=False):
    """Greedy Best-First Search evaluating nodes solely based on h(n)"""
    start = (self.state.pos_x, self.state.pos_y)
    unknowns = [] if return_home else self.get_unknown_tiles()

    # Heuristic h(n)
    def h(cx, cy):
        if return_home:
            hx, hy = self.home_pos
            return abs(cx - hx) + abs(cy - hy)
        else:
            if not unknowns: return 0
            return min(abs(cx - ux) + abs(cy - uy) for ux, uy in unknowns)
```

```
        # Queue stores tuples of (h_score, current_position, path_to_get_there)
        queue = []
        heapq.heappush(queue, (h(start[0], start[1]), start, []))
        visited = set([start])

        while queue:
            current_h, current, path = heapq.heappop(queue)
            cx, cy = current
            self.nodes_explored += 1

            # Goal Check
            if not return_home:
                if self.state.world[cx][cy] == AGENT_STATE_UNKNOWN:
                    self.route = path
                    return
            else:
                if current == self.home_pos:
                    self.route = path
                    return

            # Explore 4 directions
            for dx, dy in [(0, -1), (1, 0), (0, 1), (-1, 0)]:
                nx, ny = cx + dx, cy + dy

                if 0 <= nx < self.state.world_width and 0 <= ny <
self.state.world_height:
                    if (nx, ny) not in visited and self.state.world[nx][ny] !=
AGENT_STATE_WALL:
                        visited.add((nx, ny))
                        next_h = h(nx, ny)
                        heapq.heappush(queue, (next_h, (nx, ny), path + [(nx,
ny)]))

        self.route = []
```

How Greedy Best-First Search works:

- Greedy Best-First Search uses a min-heap priority queue ordered by the heuristic value h(n) alone.
- At each step, it expands the node that appears closest to the goal according to the heuristic function, without considering how far it has already traveled.
- The evaluation function is: f(n) = h(n)
- Starting from the agent's current position, it pushes neighbors onto the priority queue with their heuristic scores.

- When a node is popped from the queue, it is the one with the smallest h(n) value, meaning it looks the most promising based purely on estimated distance to the goal.
- Nodes are marked as visited when enqueued to prevent duplicate exploration.
- Greedy Best-First Search does NOT guarantee the shortest path. Because it ignores the cost already incurred to reach a node (g(n)), it may choose a path that looks promising heuristically but is actually longer in total steps.

2.5 A* Search Implementation

```
    def a_star_search(self, return_home=False):
        """A* Search evaluating nodes based on f(n) = g(n) + h(n)"""
        start = (self.state.pos_x, self.state.pos_y)
        unknowns = [] if return_home else self.get_unknown_tiles()

        # Heuristic h(n)
        def h(cx, cy):
            if return_home:
                hx, hy = self.home_pos
                return abs(cx - hx) + abs(cy - hy)
            else:
                if not unknowns: return 0
                return min(abs(cx - ux) + abs(cy - uy) for ux, uy in unknowns)

        # Queue stores tuples of (f_score, g_score, current_position,
path_to_get_there)
        queue = []
        g_start = 0
        h_start = h(start[0], start[1])
        heapq.heappush(queue, (g_start + h_start, g_start, start, []))

        # Track the best (lowest) g_cost to reach a specific cell
        g_costs = {start: 0}

        while queue:
            f, g, current, path = heapq.heappop(queue)
            cx, cy = current

            # Skip if we already found a better path to this node
            if g > g_costs.get(current, float('inf')):
                continue

            self.nodes_explored += 1
```

```
            # Goal Check
            if not return_home:
                if self.state.world[cx][cy] == AGENT_STATE_UNKNOWN:
                    self.route = path
                    return
            else:
                if current == self.home_pos:
                    self.route = path
                    return

            # Explore 4 directions
            for dx, dy in [(0, -1), (1, 0), (0, 1), (-1, 0)]:
                nx, ny = cx + dx, cy + dy

                if 0 <= nx < self.state.world_width and 0 <= ny <
self.state.world_height:
                    if self.state.world[nx][ny] != AGENT_STATE_WALL:
                        new_g = g + 1

                        if new_g < g_costs.get((nx, ny), float('inf')):
                            g_costs[(nx, ny)] = new_g
                            next_h = h(nx, ny)
                            heapq.heappush(queue, (new_g + next_h, new_g, (nx,
ny), path + [(nx, ny)]))

        self.route = []
```

How A* Search works:

- A* Search uses a min-heap priority queue ordered by the evaluation function f(n) = g(n) + h(n), where:
  - g(n) = the actual cost (number of steps) from the start to node n
  - h(n) = the estimated cost (Manhattan distance) from node n to the goal
- Unlike Greedy Best-First Search, A* considers both the cost already incurred and the estimated remaining cost, producing a balanced and optimal search.
- A key optimization is the g_costs dictionary, which tracks the lowest known cost to reach each node. If a node is popped with a higher g value than what's recorded, it is skipped — this avoids redundant exploration.
- When a neighbor is discovered with a lower g cost than previously recorded, the g_costs dictionary is updated and the neighbor is pushed into the priority queue with the new f value.
- A* guarantees the shortest path when the heuristic is admissible (never overestimates). Since Manhattan distance is admissible on a 4-connected grid, A* will always find the optimal path.

2.6 Key Differences Between BFS and DFS

Feature	Greedy Best-First Search	A* Search
Evaluation Function	f(n) = h(n)	f(n) = g(n) + h(n)
Data Structure	Min-heap priority queue	Min-heap priority queue
Considers Path Cost?	No (ignores g(n))	Yes (includes g(n))
Path Optimality	Not optimal	Optimal (with admissible h)
Search Behavior	Aggressively pursues closest-seeming node	Balances exploration and exploitation
Duplicate Handling	Visited set (enqueue-time)	g_costs dictionary (re-opens if cheaper path found)
Completeness	Complete (in finite spaces)	Complete
Time Complexity	O(b^m) worst case	O(b^d) with good heuristic
Space Complexity	O(b^m) worst case	O(b^d) with good heuristic

Table 2.5: Key Differences Between Best-First Search and A Search*

Where b = branching factor, d = optimal solution depth, m = maximum depth of search tree.

2.7 Move-to Logic

The move_to() method translates a path (sequence of target coordinates) into physical actions:

1. Compute target direction: Determine which direction the agent needs to face to reach the next tile.
2. Turn if necessary: If the agent is not facing the target direction, it turns right or left.
3. Move forward: Once facing the correct direction, the agent moves forward and pops the current waypoint from the route.

Each action (turn or forward) costs −1 point. Since A* returns optimal (shortest) paths, the agent needs fewer total actions when path quality matters, resulting in potentially higher scores.

3. Experimental Results

All experiments were run with dirt bias = 0.5, using a fixed random seed (1337) for reproducibility.

3.1 Performance Comparison without Obstacles (wall bias = 0.0)

	Greedy Best-First Search				A* Search			
Environment size (without obstacles)	#nodes explored	Solution length (Steps)	Score	Is it optimal?	#nodes explored	Solution length (Steps)	Score	Is it optimal?
5x5	66	65	240	No	71	65	139	Yes
10x10	344	244	2303	No	302	238	2188	Yes
15x15	871	485	7698	No	685	489	7694	Yes
20x20	1653	889	14263	No	1510	833	14319	Yes

Table 3.1: Performance Comparison without Obstacles

3.2 Performance Comparison with Obstacles (wall bias = 0.2)

	Greedy Best-First Search				A* Search			
Environment size (without obstacles)	#nodes explored	Solution length (Steps)	Score	Is it optimal?	#nodes explored	Solution length (Steps)	Score	Is it optimal?
5x5	61	69	236	No	47	56	294	Yes
10x10	356	284	1738	No	347	248	1774	Yes
15x15	743	539	4008	No	725	519	4028	Yes
20x20	1619	1069	9336	No	1456	960	9344	Yes

Table 3.2: Performance Comparison with Obstacles

3.3 Analysis

Nodes Explored

- A* explores fewer nodes than Greedy Best-First Search on most grid sizes. For example, on a 15×15 grid without obstacles, A* explores 685 nodes versus 871 for Greedy — a 21% reduction.

- This may seem counter-intuitive since Greedy is often described as expanding fewer nodes. However, Greedy's lack of path-cost awareness causes it to explore nodes along suboptimal paths that it later abandons, effectively wasting node expansions.
- A*'s g_costs pruning mechanism effectively avoids redundant exploration, keeping the total node count lower in practice.
- With obstacles (wall bias = 0.2), both algorithms explore fewer nodes because walls reduce the number of reachable tiles.

Solution Length (Steps)

- A* produces shorter solutions in most cases, particularly on larger grids and with obstacles. For 20×20 with obstacles, A* completes in 960 steps while Greedy takes 1,069 steps — a 10% improvement.
- On smaller grids (5×5 without obstacles), both algorithms produce the same step count (65), because the grid is small enough that suboptimal paths by Greedy still end up similar in length.
- The advantage of A* grows with grid size and complexity, as deeper searches compound the effect of path-cost awareness.

Score

- Both algorithms achieve comparable high scores, with A* slightly outperforming Greedy on larger grids. For 20×20 without obstacles: A* scores 14,319 vs Greedy's 14,263.
- With obstacles, the gap is more pronounced: A* scores 9,344 vs 9,336 on 20×20, and 249 vs 236 on 5×5.
- On smaller grids without obstacles, Greedy occasionally scores higher (e.g., 240 vs 139 on 5×5). This occurs because the random initialization phase and different search orders cause the two algorithms to encounter slightly different dirt patterns, meaning Greedy may stumble upon more dirt tiles during its less-optimal exploration.
- The score differences are relatively small between the two algorithms because both are informed search strategies that use the same heuristic — unlike the dramatic differences seen between uninformed searches like BFS and DFS.

Impact of Obstacles

- Obstacles reduce the number of accessible tiles and dirt tiles, lowering the maximum achievable score.
- With obstacles, A*'s advantage becomes more visible because walls create dead ends and detours. Greedy may dive into dead-end corridors that heuristically look promising but require costly backtracking.

- A* avoids this trap by accounting for the actual cost traveled, making it more resistant to maze-like environments.

Optimality

- A* is optimal: With an admissible and consistent heuristic (Manhattan distance on a 4-connected grid), A* is guaranteed to find the path with the minimum number of steps. Each path found by A* is provably the shortest possible path to the target.
- Greedy Best-First Search is NOT optimal: It only considers the heuristic estimate and ignores the actual cost, so it may return paths that are longer than necessary. However, it often finds reasonably good paths quickly, making it a practical choice when optimality is not required.

Why Both Algorithms Perform Similarly

Unlike BFS vs DFS (where DFS can fail catastrophically), Greedy and A* perform similarly because:

1. Shared heuristic: Both use the same Manhattan distance heuristic, guiding them toward goals efficiently.
2. Priority queue: Both use min-heaps, ensuring the most promising node is expanded first.
3. Informed search: Both leverage domain knowledge (distance estimates), unlike uninformed searches that explore blindly.
4. Grid topology: In simple open grids, the heuristic is very accurate (Manhattan distance = actual distance when no walls block), so Greedy's greedy choices happen to coincide with optimal choices most of the time.

The differences emerge most in complex environments with obstacles, where the heuristic can be misleading and only A*'s path-cost consideration prevents suboptimal decisions.

4. Conclusion

The experimental results demonstrate that A* Search and Greedy Best-First Search are both effective informed search strategies for the vacuum agent problem, with A* having a theoretical and practical edge:

1. Fewer nodes explored — A* explores fewer nodes on average due to its g_costs pruning mechanism, avoiding redundant path exploration.
2. Shorter solution paths — A* guarantees optimal paths, leading to fewer total actions especially in complex environments.
3. Higher scores — A*'s optimal paths translate to slightly higher scores, though the difference is modest compared to uninformed search comparisons.

4. Optimality — A* is provably optimal with an admissible heuristic, while Greedy Best-First Search offers no optimality guarantee.
5. Robustness — A* maintains strong performance even with obstacles and dead ends, while Greedy may suffer from heuristic traps.
6. Both complete successfully — Unlike DFS which can fail on larger grids, both informed search algorithms complete all scenarios and return home within the iteration limit.

The key takeaway is that the quality of the heuristic function is crucial for informed search. When the heuristic is accurate (as Manhattan distance is on open grids), both algorithms perform similarly. When the heuristic can be misleading (environments with obstacles), A*'s consideration of actual path cost (g(n)) provides a meaningful advantage, making it the superior choice for guaranteed optimal behavior.